PROGRAMACIÓN WEB · GRUPO C · 7.º SEMESTRE

PLANEACIÓN Y ORGANIZACIÓN DEL PROYECTO FINAL DE SOFTWARE

UmaRacing Online: videojuego de carreras en el navegador con plataforma web y publicidad simulada

Equipo 01

Proyecto asignado	Proyecto 3 — Videojuego con publicidad
Materia	Programación Web
Docente	Francisco Hunahpu Sahagún González
Integrantes	Javier Antonio Rodríguez Morales Marco Antonio Muñoz Malpica José Oliver Guzmán Camacho
Grupo	C
Fecha de este plan	07/10/2026
Periodo planeado	07/10/2026 al 06/11/2026 (fecha de entrega indicada por el docente)
Repositorio Git	https://github.com/Perrullo/Uma-Racing

Documento de planeación (línea base). Las evidencias mencionadas son **esperadas**: se generarán durante el desarrollo y no se presentan como resultados realizados.

Índice

1. Presentación y alcance del proyecto . . . 3
1.1 Proyecto e integrantes . . . 3
1.2 Problema que se atenderá . . . 3
1.3 Usuarios principales . . . 3
1.4 Objetivo general . . . 3
1.5 Funciones que incluirá la versión final . . . 3
1.6 Exclusiones y restricciones . . . 4
2. Identificación de requisitos y entregables . . . 5
2.1 Entregables del proyecto . . . 7
3. Desglose de actividades y responsabilidades . . . 8
3.1 Responsabilidades y carga por integrante . . . 12
4. Cronograma de trabajo . . . 13
4.1 Diagrama de Gantt por actividad . . . 15
5. Recursos y decisiones técnicas . . . 17
6. Riesgos y seguimiento . . . 18
6.1 Registro de riesgos . . . 18
6.2 Seguimiento del avance . . . 18
7. Plan de pruebas y entrega . . . 20
7.1 Estrategia de pruebas . . . 20
7.2 Lista de verificación para la entrega final . . . 20

1. Presentación y alcance del proyecto

1.1 Proyecto e integrantes

Nombre del proyecto: UmaRacing Online. **Integrantes:** Javier Antonio Rodríguez Morales, Marco Antonio Muñoz Malpica y José Oliver Guzmán Camacho (Equipo 01). **Proyecto asignado:** Proyecto 3 — Videojuego con publicidad.

1.2 Problema que se atenderá

UmaRacing es un juego de carreras para dos personas en el mismo teclado, hecho en Python con pygame. Hoy solo funciona en escritorio: hay que instalar Python para jugarlo, no guarda partidas, puntajes ni progreso, no tiene cuentas de usuario y no ofrece ninguna forma de sostener su operación ni de medir su uso. Además, si las recompensas se calcularan en el cliente, cualquier jugador podría alterarlas desde el navegador. El proyecto convertirá el juego en un sistema web en el que el servidor registre y valide cada partida, compra y recompensa.

1.3 Usuarios principales

- **Jugador (rol “jugador”):** crea su cuenta, juega en el navegador como Jugador 1, gana y gasta monedas virtuales, desbloquea corredoras y escenarios, ve anuncios con recompensa y compite en el ranking.
- **Invitado:** juega como Jugador 2 en el mismo teclado, sin cuenta.
- **Administrador (rol “administrador”):** gestiona usuarios, artículos, anuncios, logros, paquetes de monedas y premios; consulta el dashboard, los reportes y la bitácora; y reembolsa compras.

1.4 Objetivo general

Desarrollar, del 07/10/2026 al 06/11/2026, un sistema web cliente-servidor que ejecute el videojuego UmaRacing en el navegador y que registre y valide en el backend partidas, puntajes, compras, recompensas y publicidad simulada, con persistencia en MariaDB, control de acceso por roles, reportes y estadísticas, sin modificar la jugabilidad original del juego.

1.5 Funciones que incluirá la versión final

- Registro, inicio y cierre de sesión con contraseñas cifradas (bcrypt) y token JWT revocable; perfil del jugador.
- Juego en el navegador (WebAssembly) con corredoras bloqueadas o desbloqueadas según el inventario.
- Registro de partidas validado por el servidor (antitrampa), puntaje, monedas, logros automáticos y ranking por periodo.
- Tienda de corredoras y escenarios con compra transaccional, inventario, libro mayor de monedas y reembolsos.
- Publicidad simulada: ver anuncio → completar evento → recibir recompensa validada por el servidor.
- Monetización simulada: compra de monedas con Google Pay en modo de prueba, monedas por tiempo de juego, apuestas con monedas virtuales y gachapón con probabilidades públicas.

- Administración con CRUD de usuarios, artículos, anuncios, logros, paquetes y premios; búsqueda, filtros, ordenamiento y paginación.
- Dashboard por rol, 4 reportes con rango de fechas, gráficas dinámicas, exportación CSV y bitácora de operaciones.

1.6 Exclusiones y restricciones

Ninguna exclusión elimina requisitos obligatorios de los documentos base; todos se cubren en la matriz de la sección 2.

- **No se cobrará dinero real:** Google Pay se usará solo en su entorno de pruebas (TEST). Las monedas son virtuales y no se pueden retirar.
- **No habrá modo en línea entre equipos distintos:** las carreras seguirán siendo locales, en un mismo teclado, como en el juego original.
- **No se integrarán redes publicitarias reales:** los anuncios serán simulados.
- **No se modificará la jugabilidad del juego original**; solo se agregarán conexiones con la plataforma.
- **No se desarrollará una app móvil nativa;** el portal será responsivo.

Restricciones de tiempo: 31 días naturales, con una disponibilidad estimada de 20 a 25 horas por semana por integrante, compartidas con otras materias. **Restricciones de conocimientos:** el equipo tiene poca experiencia con WebAssembly (pygbag), FastAPI y Docker, por lo que se reserva tiempo de aprendizaje en las primeras actividades de cada tema. **Restricciones de infraestructura:** desarrollo en computadoras personales con XAMPP (MariaDB) y despliegue en un servidor con Docker y Portainer; el juego web requiere internet para descargar su intérprete desde el CDN de pygame-web.

2. Identificación de requisitos y entregables

La matriz relaciona cada requisito obligatorio de los *Requerimientos de Proyectos Finales de Software* (generales y del Proyecto 3) y de los *Lineamientos de Documentación* con el módulo donde se implementará, las actividades de la sección 3, el responsable y la evidencia esperada para comprobar su cumplimiento.

Tabla 1. Matriz de requisitos obligatorios

ID	Requisito obligatorio	Módulo	Actividades	Responsable	Evidencia esperada
Requerimientos generales					
R01	Arquitectura cliente-servidor: frontend, API REST, backend y BD; el frontend no accede a la BD	Arquitectura	A07, A17	Javier R.	Diagrama de arquitectura; backend separado en capas; credenciales solo en el servidor
R02	BD real con al menos 6 tablas relacionadas, normalizada hasta 3FN, con PK, FK, restricciones e índices	Base de datos	A09, A12–A15	Javier R. (con Marco M. y José G.)	Diagrama ER, diccionario de datos y `schema.sql` que crea la BD desde cero
R03	Script SQL y datos de prueba	Base de datos	A16	José G.	`seed.sql` y `seed_demo.sql` cargados sin errores
R04	API REST con métodos HTTP correctos, JSON y códigos de estado (200, 201, 400, 401, 403, 404, 409, 422, 500)	API	A10, A17	Javier R.	Diseño de la API y respuestas de error uniformes comprobadas en pruebas
R05	Al menos 12 endpoints funcionales	API	A18–A27	Todos	Lista de endpoints en Swagger (`/docs`)
R06	Al menos 3 módulos principales con CRUD	Administración	A20, A22, A23, A24, A26	Javier R., José G.	CRUD de artículos, anuncios, usuarios, paquetes y premios funcionando en pantallas de administración
R07	Autenticación: login, logout, control de sesión, contraseñas con hash y rutas protegidas	Autenticación	A18, A30	Javier R.	Hash bcrypt en la BD; token revocado tras logout; 401 sin token
R08	Al menos 2 roles con permisos verificados en el backend	Autenticación	A18	Javier R.	Prueba: un jugador recibe 403 en rutas de administrador
R09	Dashboard con tarjetas, contadores, tablas, gráficas e indicadores	Panel	A27, A37	José G.	Dashboard por rol con datos reales y alertas
R10	Al menos 5 reglas de negocio reales	Varios	A04, A20–A26	José G. (definición); dueños de módulo	Tabla de reglas con una prueba asociada a cada una
R11	Operación transaccional (COMMIT / ROLLBACK)	Tienda	A20	Javier R.	Compra con folio: prueba de éxito y de saldo insuficiente sin cambios
R12	Buscador dinámico con datos reales	Listados	A19, A33	José G.	Búsqueda por texto en tablas de administración e historial
R13	Filtros (al menos un módulo)	Listados	A19	José G.	Filtros por estado, tipo, rol y fechas
R14	Ordenamiento en tablas principales	Listados	A19, A29	José G., Marco M.	Columnas ordenables con lista blanca en el backend
R15	Paginación con formato estándar	Listados	A19, A29	José G., Marco M.	`?page=&limit=` → `{pagina, limite, total, totalPaginas, datos}`

ID	Requisito obligatorio	Módulo	Actividades	Responsable	Evidencia esperada
R16	Historial de operaciones (fecha, usuario, operación, registro, estado anterior y nuevo)	Bitácora	A23	José G.	Pantalla de bitácora con filtros
R17	Baja lógica de información importante	Administración	A20, A23	Javier R., José G.	Campos `activo` / `eliminado_en`; el historial se conserva
R18	Al menos 2 reportes con datos de la BD; uno con fecha inicial y final	Reportes	A27, A37	José G.	Reportes con rango de fechas y exportación CSV
R19	Al menos 2 gráficas dinámicas con datos de la API	Reportes	A37	José G.	Gráficas Chart.js alimentadas por la API
R20	Validaciones en frontend y backend	Todos	A18–A27, A29–A37	Todos	Formularios con mensajes por campo; 422 con detalle desde la API
R21	Manejo de errores con mensajes comprensibles, sin SQL, trazas ni credenciales	API	A17	Javier R.	Respuestas de error JSON; prueba con BD caída (503)
R22	Seguridad: consultas parametrizadas, protección contra SQL Injection, endpoints protegidos, sanitización	API	A17, A18	Javier R.	Prueba de inyección SQL sin efecto; revisión de consultas
R23	Variables de entorno (`.env` fuera del repositorio y `.env.example`)	Configuración	A02, A17	Javier R.	`.env.example` versionado; `.env` en `.gitignore`
R24	Diseño responsivo (laptop, PC, tablet y, preferentemente, smartphone)	Frontend	A29, A41	Marco M.	Capturas a 1366, 820 y 390 px de ancho
R25	Estados de interfaz (cargando, sin resultados, error, éxito) y confirmaciones en operaciones críticas	Frontend	A29	Marco M.	Componentes de estado y modal de confirmación usados en todas las pantallas
R26	Navegación coherente (login → dashboard → módulos)	Frontend	A08, A29	Marco M.	Mapa de navegación y menú por rol
R27	Uso continuo de Git con participación identificable de cada integrante	Repositorio	A02, A39, A45	Todos	Commits semanales de los 3 integrantes en ramas `feature/*` integradas por pull request
R28	README (descripción, integrantes, tecnologías, instalación, configuración, usuarios de prueba, endpoints, capturas)	Documentación	A46	Javier R.	`README.md` verificado por otro integrante
R29	Documentación de API (método, URL, parámetros, body, respuesta, códigos y permisos)	Documentación	A47	Javier R.	Swagger en `/docs` y `docs/api.md`
R30	Instrucciones de instalación y ejecución	Documentación	A46	Javier R.	Sección de instalación del README probada en una computadora limpia
R31	Documentación final en PDF con la estructura obligatoria de 16 secciones	Documentación	A45, A48, A49	Todos (José G. ensambla)	PDF con portada, índice, páginas numeradas y checklist completo
Requerimientos específicos del Proyecto 3 — Videojuego con publicidad					
P01	Usuarios y perfiles; registrar usuario e iniciar sesión	Perfil	A18, A30	Javier R.	Pantallas de registro, login y perfil conectadas a la API
P02	Registrar partidas y puntuación; guardar progreso	Partidas	A21, A38	Marco M.	Partida jugada en el navegador registrada en la BD con su puntaje
P03	Consultar ranking	Ranking	A21, A32	Marco M.	Ranking por periodo y criterio

ID	Requisito obligatorio	Módulo	Actividades	Responsable	Evidencia esperada
P04	Registrar logros	Logros	A21, A32	Marco M.	Logros otorgados automáticamente al cumplir la meta
P05	Publicidad simulada: ver anuncio → completar evento → recibir recompensa	Publicidad	A22, A33	José G.	Pantalla de anuncios con recompensa en monedas
P06	El servidor valida las recompensas (no basta con modificar una variable en JavaScript)	Publicidad y partidas	A21, A22	Marco M., José G.	Pruebas: reclamar un anuncio antes de tiempo y reportar un resultado imposible son rechazados
P07	Recompensas (monedas, objetos virtuales)	Tienda y monedero	A20, A24–A26	Javier R.	Libro mayor de monedas con cada movimiento
P08	Mostrar estadísticas	Reportes	A27, A37	José G.	Estadísticas del jugador y reportes del administrador
P09	El videojuego incorpora un sistema backend	Juego	A28, A38	Marco M.	El juego consulta y registra datos solo a través de la API

2.1 Entregables del proyecto

Tabla 2. Entregables principales

Entregable	Contenido	Fecha planeada
Plan de trabajo (este documento)	Alcance, matriz de requisitos, actividades, cronograma, Gantt, recursos, riesgos y plan de pruebas	09/10/2026
Documentos de análisis y diseño	Requerimientos, reglas de negocio, arquitectura, wireframes, modelo ER, diccionario de datos y diseño de API en `docs/`	14/10/2026
Base de datos	`database/schema.sql`, `seed.sql`, `seed_demo.sql`	16/10/2026
MVP integrado	Autenticación, tienda, partidas, publicidad y administración funcionando de punta a punta	23/10/2026
Sistema completo (congelamiento)	Todos los módulos integrados en `develop`	30/10/2026
Pruebas	Suite automatizada, matriz de pruebas y evidencias manuales	02/11/2026
Sistema desplegado	Stack Docker en Portainer con URL pública	02/11/2026
Repositorio final	Historial de participación, README, documentación de API y etiqueta `v1.0.0`	06/11/2026
Documentación final en PDF	16 secciones de los lineamientos	06/11/2026
Demostración	Presentación del sistema funcionando	06/11/2026

3. Desglose de actividades y responsabilidades

El trabajo se divide en 51 actividades verificables, con un total estimado de 376 horas. La responsabilidad se reparte **por módulos de punta a punta**: el responsable de un módulo diseña sus tablas, programa su API y sus pantallas, escribe sus pruebas y redacta su sección del PDF. Así, cada integrante tiene participación identificable en el desarrollo y en la documentación.

Tabla 3. Actividades, responsables, duración, dependencias y criterio de terminado

ID	Descripción	Responsable / apoyo	Duración	Depende de	Entregable	Criterio de terminado
Fase 0 · Planeación y análisis						
A01	Revisar lineamientos y requerimientos y elaborar la lista de requisitos obligatorios	**Javier R.** Apoyo: Marco M., José G.	4 h 2 días	—	Matriz de requisitos (sección 2)	Cada requisito obligatorio tiene módulo, actividad y responsable.
A02	Configurar el repositorio (ramas main, develop y feature/*, .gitignore, .env.example) y el tablero de tareas	**Javier R.**	2 h 1 día	—	Repositorio en GitHub y tablero GitHub Projects	Los 3 integrantes tienen acceso y un commit propio en develop.
A03	Definir requerimientos funcionales y no funcionales numerados (RF-xx, RNF-xx)	**Marco M.** Apoyo: Javier R., José G.	5 h 2 días	A01	docs/requerimientos.md	RF y RNF numerados y revisados por los tres integrantes.
A04	Definir al menos 5 reglas de negocio y los criterios de aceptación de cada módulo	**José G.** Apoyo: Javier R., Marco M.	4 h 2 días	A01	docs/requerimientos.md	Al menos 5 RN verificables; cada módulo con condiciones observables.
A05	Describir el modelo de negocio y el contexto de uso	**José G.**	3 h 1 día	A01	Borrador de la sección 4 del PDF	Incluye los 9 elementos mínimos del lineamiento 7.1.
A06	Elaborar este plan: actividades, cronograma, Gantt, riesgos y plan de pruebas	**José G.** Apoyo: Javier R., Marco M.	6 h 3 días	A01	Planeacion_Proyecto_Equipo01.pdf	Entregado en Teams (Proyecto Unidad 2).
Fase 1 · Diseño						
A07	Diagrama de arquitectura cliente-servidor y estructura de carpetas	**Javier R.**	3 h 2 días	A03	docs/diagrams/arquitectura	Muestra usuario, frontend, API, backend, BD y servicios externos.
A08	Wireframes de login, dashboard, tienda, formulario, detalle y reportes	**Marco M.** Apoyo: José G.	6 h 3 días	A03	docs/diagrams/wireframes	Las 6 pantallas mínimas del lineamiento 11.1.
A09	Modelo entidad-relación en 3FN y diccionario de datos	**Javier R.** Apoyo: Marco M., José G.	6 h 3 días	A03, A04	docs/database.md y diagrama ER	Al menos 6 tablas relacionadas, con PK, FK y cardinalidades; 3FN justificada.
A10	Lista de endpoints con método, entrada, respuesta, códigos HTTP y rol	**Javier R.** Apoyo: Marco M., José G.	5 h 3 días	A09	docs/api.md (borrador)	Al menos 12 endpoints con permiso por rol.
A11	Paleta, tipografía, componentes y estados de interfaz	**Marco M.**	4 h 3 días	A08	docs/ux.md	Contraste validado; estados de carga, vacío, error y éxito definidos.

ID	Descripción	Responsable / apoyo	Duración	Depende de	Entregable	Criterio de terminado
Fase 2 · Base de datos						
A12	schema.sql: roles, usuarios, sesiones, perfiles y catálogo de artículos	**Javier R.**	5 h 2 días	A09	database/schema.sql	La BD se crea desde cero sin errores en MariaDB.
A13	Tablas de compras, inventario, movimientos de monedas, paquetes y órdenes de pago	**Javier R.**	4 h 2 días	A12	database/schema.sql	FK, UNIQUE y CHECK de saldo definidos.
A14	Tablas de partidas, logros, sesiones de juego, apuestas y atributos de carrera	**Marco M.**	4 h 2 días	A12	database/schema.sql	FK e índices por usuario y fecha definidos.
A15	Tablas de anuncios, visualizaciones, gachapón y bitácora	**José G.**	4 h 2 días	A12	database/schema.sql	FK y CHECK de probabilidades y duraciones definidos.
A16	Datos semilla y de demostración (seed.sql, seed_demo.sql) y script de reinicio	**José G.** Apoyo: Javier R.	5 h 2 días	A13, A14, A15	database/seed.sql, seed_demo.sql	Se cargan sin errores; hay cuentas de prueba para cada rol.
Fase 3 · Backend y API						
A17	Configuración por .env, consultas parametrizadas, transacciones y manejo uniforme de errores	**Javier R.**	8 h 2 días	A12	backend/app (núcleo)	/api/salud responde; los errores son JSON sin SQL ni trazas.
A18	Registro, login, logout con revocación, bcrypt, JWT y permisos por rol	**Javier R.**	10 h 4 días	A17	/api/auth/*, security.py	Sin token responde 401; con rol incorrecto, 403; contraseñas con hash.
A19	Búsqueda, filtros, ordenamiento con lista blanca y paginación estándar	**José G.**	5 h 3 días	A17	services/consultas.py	Respuesta {pagina, limite, total, totalPaginas, datos}.
A20	CRUD de artículos, compra transaccional con folio, inventario, libro mayor y reembolso	**Javier R.** Apoyo: Marco M.	16 h 4 días	A13, A18, A19	/api/items, /api/compras	Compra con COMMIT y ROLLBACK comprobados.
A21	Registro de partidas, validación antitrampa, puntaje, monedas, logros automáticos y ranking	**Marco M.** Apoyo: Javier R.	16 h 4 días	A14, A18	/api/partidas, /api/logros, /api/ranking	Un resultado imposible se rechaza; los logros se otorgan solos.
A22	Publicidad simulada con recompensa validada por el servidor y CRUD de anuncios	**José G.**	8 h 2 días	A15, A18	/api/anuncios	No paga si el anuncio no se vio completo; límite diario.
A23	CRUD de usuarios (baja lógica, ajuste de monedas) y bitácora de operaciones	**José G.** Apoyo: Javier R.	10 h 3 días	A18, A19	/api/usuarios, /api/bitacora	Operaciones críticas registradas con estado anterior y nuevo.
A24	Paquetes de monedas y compra con Google Pay en modo TEST	**Javier R.**	10 h 4 días	A20	/api/paquetes, /api/pagos	La orden se paga una sola vez y vence en 15 minutos.
A25	Monedas por tiempo de juego activo y apuestas (carrera simulada y reto de tiempo)	**Marco M.**	14 h 5 días	A20, A21	/api/tiempo-juego, /api/apuestas	Límites de juego responsable; el resultado lo decide el servidor.
A26	Gachapón con probabilidades públicas y CRUD de premios	**José G.**	10 h 3 días	A20	/api/gacha	Las probabilidades suman 100 %; garantía de premio épico.
A27	Dashboard por rol y 4 reportes con rango de fechas	**José G.** Apoyo: Javier R., Marco M.	12 h 4 días	A20, A21, A22, A23	/api/dashboard, /api/reportes/*	Datos tomados de la BD; un rango de fechas inválido responde 422.

ID	Descripción	Responsable / apoyo	Duración	Depende de	Entregable	Criterio de terminado
Fase 4 · Frontend						
A28	Compilar el juego a WebAssembly con pygbag y una plantilla propia	**Marco M.**	6 h 3 días	—	game/build/web, plantilla_juego/	El juego carga en el navegador sin cambiar su jugabilidad.
A29	Layout y menú por rol, cliente de API, tabla y CRUD reutilizables, estados, confirmaciones y diseño responsivo	**Marco M.** Apoyo: José G.	14 h 5 días	A10, A11	frontend/js, frontend/css	Navegación coherente a 1366, 820 y 390 px.
A30	Pantallas de login, registro y perfil	**Javier R.** Apoyo: Marco M.	6 h 2 días	A18, A29	login.html, registro.html, perfil.html	Validación en el frontend y mensajes del servidor visibles.
A31	Pantallas de tienda e historial de compras y movimientos	**Javier R.**	8 h 2 días	A20, A29	tienda.html, historial	Compra con confirmación y saldo actualizado sin recargar.
A32	Pantallas de jugar, historial de partidas, ranking y logros	**Marco M.**	10 h 2 días	A21, A29	jugar.html, ranking.html, logros.html	Datos de la API con filtros y paginación.
A33	Pantallas de anuncios, administración (usuarios, artículos, anuncios, logros) y bitácora	**José G.** Apoyo: Marco M.	12 h 3 días	A22, A23, A29	anuncios.html, admin/*.html	CRUD completo con confirmaciones; un jugador no puede acceder.
A34	Compra de monedas con Google Pay y administración de paquetes	**Javier R.**	6 h 2 días	A24	monedas.html, admin/paquetes.html	Pago de prueba acreditado y visible en el historial.
A35	Apuestas con carrera animada y tiempo de juego en jugar.html	**Marco M.**	8 h 2 días	A25	apuestas.html	La animación repite la carrera decidida por el servidor.
A36	Pantalla del gachapón y administración de premios	**José G.**	6 h 2 días	A26	gacha.html, admin/gacha.html	Muestra probabilidades públicas y contador de garantía.
A37	Dashboard y reportes con Chart.js, filtro de fechas y exportación CSV	**José G.** Apoyo: Marco M.	8 h 2 días	A27, A29	dashboard.html, admin/reportes.html	Al menos 2 gráficas dinámicas con datos de la API.
Fase 5 · Integración						
A38	Candados según inventario, registro de partidas y latidos de tiempo de juego	**Marco M.** Apoyo: Javier R.	8 h 2 días	A21, A28, A32	game/conexion.py, jugar.html	Una partida jugada en el navegador queda registrada en la BD.
A39	Integrar ramas en develop con revisión cruzada (pull requests) y prueba de humo	**Javier R.** Apoyo: Marco M., José G.	6 h 2 días	A30–A38	develop estable (congelamiento de funciones)	El flujo punta a punta funciona sin errores.
Fase 6 · Pruebas						
A40	Pruebas de API con pytest y BD aislada: cada integrante cubre sus módulos; José G. coordina la suite	**Equipo**	16 h 11 días	A20–A27	backend/tests	Cubren éxito, datos inválidos, permisos, inexistente, duplicado, regla crítica y transacción.
A41	Pruebas de diseño responsivo, estados de interfaz, error de conexión y flujo punta a punta con capturas	**Marco M.** Apoyo: José G.	6 h 2 días	A39	Capturas en docs/	Cada caso manual con evidencia de captura.
A42	Matriz de pruebas y registro de defectos	**José G.**	4 h 2 días	A40, A41	docs/pruebas.md	Cada caso tiene su resultado obtenido y su estado.

ID	Descripción	Responsable / apoyo	Duración	Depende de	Entregable	Criterio de terminado
Fase 7 · Correcciones						
A43	Corregir defectos y volver a ejecutar las pruebas	**Equipo**	12 h 3 días	A42	Commits fix: en develop	Ningún defecto crítico abierto; la suite completa aprueba.
Fase 8 · Despliegue						
A44	Dockerfiles y docker-compose (db, api, nginx) y despliegue en Portainer desde GitHub	**José G.** Apoyo: Javier R.	10 h 6 días	A27	Dockerfile, docker-compose.yml	Sistema accesible en una URL pública.
Fase 9 · Documentación						
A45	Cada viernes, cada responsable actualiza la documentación de sus módulos y registra los cambios al cronograma	**Equipo**	8 h 22 días	—	docs/ actualizado	Un commit docs: por integrante cada semana.
A46	README con instalación, configuración, variables de entorno, usuarios de prueba y endpoints	**Javier R.** Apoyo: José G.	4 h 15 días	A18	README.md	Una persona ajena instala y ejecuta el sistema siguiéndolo.
A47	Swagger/OpenAPI y docs/api.md con parámetros, cuerpos, respuestas, códigos y permisos	**Javier R.** Apoyo: Marco M., José G.	5 h 8 días	A20–A27	/docs y docs/api.md	Todos los endpoints documentados con su rol.
A48	Redactar el PDF final por secciones: Javier (7, 9, 10, 12), Marco (3, 4, 8, 11), José (2, 5, 13, 14, 15, 16)	**Equipo**	15 h 6 días	A39	Borrador del PDF final	Las 16 secciones obligatorias redactadas.
A49	Ensamblar, revisar en cruzado y aplicar el checklist del PDF final	**José G.** Apoyo: Javier R., Marco M.	4 h 2 días	A48	UmaRacing_Documentacion.pdf	Checklist de los lineamientos completo.
Fase 10 · Entrega						
A50	Ensayar la demostración: guion, cuentas y datos de prueba	**Equipo**	3 h 1 día	A43, A44	Guion de la demostración	La demo se completa en el tiempo asignado.
A51	Etiqueta v1.0.0, push a GitHub, entrega del PDF y demostración	**José G.** Apoyo: Javier R., Marco M.	2 h 1 día	A49, A50	Release v1.0.0 y PDF en Teams	Entregado en la fecha indicada por el docente.

3.1 Responsabilidades y carga por integrante

Tabla 4. Distribución por módulos y carga estimada

Integrante	Módulos y áreas a cargo	Actividades como responsable	Actividades de apoyo	Horas estimadas	%
Javier Antonio Rodríguez Morales	Núcleo (BD, API, autenticación), tienda y monedero, Google Pay, README y documentación de API	17	11	126 h	34 %
Marco Antonio Muñoz Malpica	Juego en el navegador, partidas, logros, ranking, tiempo de juego, apuestas, base del frontend y UX/UI	12	14	119 h	32 %
José Oliver Guzmán Camacho	Publicidad, gachapón, usuarios y bitácora, búsqueda y paginación, dashboard y reportes, pruebas, despliegue y ensamble del PDF	17	10	131 h	35 %
Total estimado				**376 h**	100 %

Las actividades de “Todo el equipo” (corrección de defectos, documentación semanal, redacción del PDF y ensayo de la demo) se reparten en partes iguales. Además de las horas como responsable, cada integrante apoya actividades de otros módulos (revisiones de código, pruebas cruzadas y diseño de sus tablas).

Tabla 5. Secciones del PDF final a cargo de cada integrante

Integrante	Secciones de la documentación final
Javier Antonio Rodríguez Morales	7. Arquitectura · 9. Diseño de base de datos · 10. Diseño de API · 12. Seguridad · README y documentación de API
Marco Antonio Muñoz Malpica	3. Problema, objetivos y alcance · 4. Modelo de negocio · 8. Diseño UX/UI · 11. Módulos y lógica de negocio
José Oliver Guzmán Camacho	1. Portada · 2. Resumen ejecutivo · 5. Planeación (seguimiento real contra planeado) · 13. Pruebas · 14. Despliegue · 15. Resultados · 16. Anexos · ensamble final

4. Cronograma de trabajo

El cronograma abarca del **miércoles 07/10/2026** a la **entrega del viernes 06/11/2026** e incluye las fases de análisis, diseño, base de datos, backend, frontend, integración, pruebas, correcciones, despliegue, documentación y entrega. La documentación se actualiza cada viernes durante el desarrollo (A45). Cada hito cae en viernes y se asocia con un resultado verificable. Del 02/11 al 05/11 se reservan días para corregir errores e integrar el trabajo del equipo antes de la entrega.

Tabla 6. Cronograma por fase

Fase o actividad	Inicio	Fin	Responsable	Entregable estimado	Evidencia esperada	Estado
Planeación y análisis A01–A06	07/10/2026	09/10/2026	Javier R., Marco M., José G.	Matriz de requisitos, RF/RNF, reglas de negocio, modelo de negocio y plan de trabajo	docs/requerimientos.md y Planeacion_Proyecto_Equipo01.pdf en Teams	Planeado
Diseño A07–A11	10/10/2026	14/10/2026	Javier R., Marco M.	Arquitectura, wireframes, modelo ER, diccionario de datos, diseño de API y guía visual	docs/diagrams/, docs/database.md, docs/api.md y docs/ux.md en el repositorio	Planeado
Base de datos A12–A16	13/10/2026	16/10/2026	Javier R., Marco M., José G.	Script de creación de la BD (al menos 6 tablas en 3FN) y datos semilla	schema.sql y seed.sql ejecutados sin errores en MariaDB	Planeado
Backend y API A17–A27	15/10/2026	29/10/2026	Javier R., Marco M., José G.	API REST con autenticación, roles, reglas de negocio, transacciones y reportes	Endpoints visibles en Swagger (/docs) y pruebas de cada módulo aprobadas	Planeado
Frontend A28–A37	13/10/2026	30/10/2026	Javier R., Marco M., José G.	Juego en el navegador y pantallas del jugador y del administrador	Pantallas funcionando contra la API; capturas en docs/	Planeado
Integración A38–A39	22/10/2026	30/10/2026	Javier R., Marco M.	Juego conectado con la API y ramas integradas en develop	Partida jugada en el navegador registrada en la BD; develop estable	Planeado
Pruebas A40–A42	21/10/2026	02/11/2026	Marco M., José G., Equipo	Suite automatizada, pruebas manuales y matriz de pruebas	backend/tests aprobado; docs/pruebas.md con resultados	Planeado
Correcciones A43–A43	02/11/2026	04/11/2026	Equipo	Defectos corregidos	Commits fix: en develop y suite completa aprobada	Planeado
Despliegue A44–A44	28/10/2026	02/11/2026	José G.	Sistema desplegado con Docker y Portainer	URL pública funcionando	Planeado
Documentación A45–A49	09/10/2026	05/11/2026	Javier R., José G., Equipo	README, documentación de API y PDF final actualizados	README.md, /docs, docs/api.md y PDF con el checklist completo	Planeado

Fase o actividad	Inicio	Fin	Responsable	Entregable estimado	Evidencia esperada	Estado
Entrega A50–A51	05/11/2026	06/11/2026	José G., Equipo	Versión v1.0.0, PDF y demostración	Release en GitHub y entrega en Teams	Planeado

Tabla 7. Hitos de control

Hito	Fecha	Día	Resultado verificable	Estado
H1	09/10/2026	viernes	Plan de trabajo entregado; requisitos, reglas y modelo de negocio definidos	Planeado
H2	16/10/2026	viernes	BD creada con datos semilla, núcleo de la API y juego en el navegador	Planeado
H3	23/10/2026	viernes	MVP: autenticación, tienda, partidas, publicidad y administración integradas	Planeado
H4	30/10/2026	viernes	Congelamiento de funciones: todos los módulos integrados en develop	Planeado
H5	04/11/2026	miércoles	Defectos críticos corregidos, sistema desplegado y PDF en revisión	Planeado
H6	06/11/2026	viernes	Entrega final: sistema, repositorio, PDF y demostración	Planeado

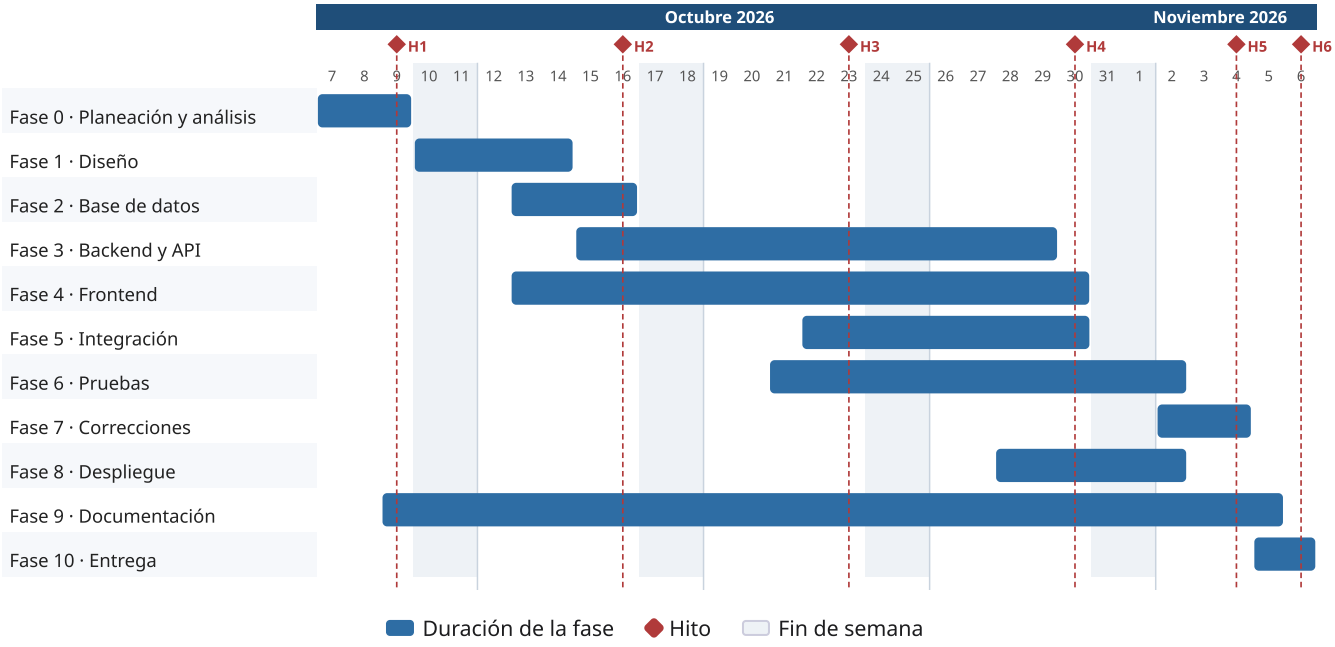


Figura 1. *Diagrama de Gantt por fase, con los hitos de control (H1–H6).*

4.1 Diagrama de Gantt por actividad

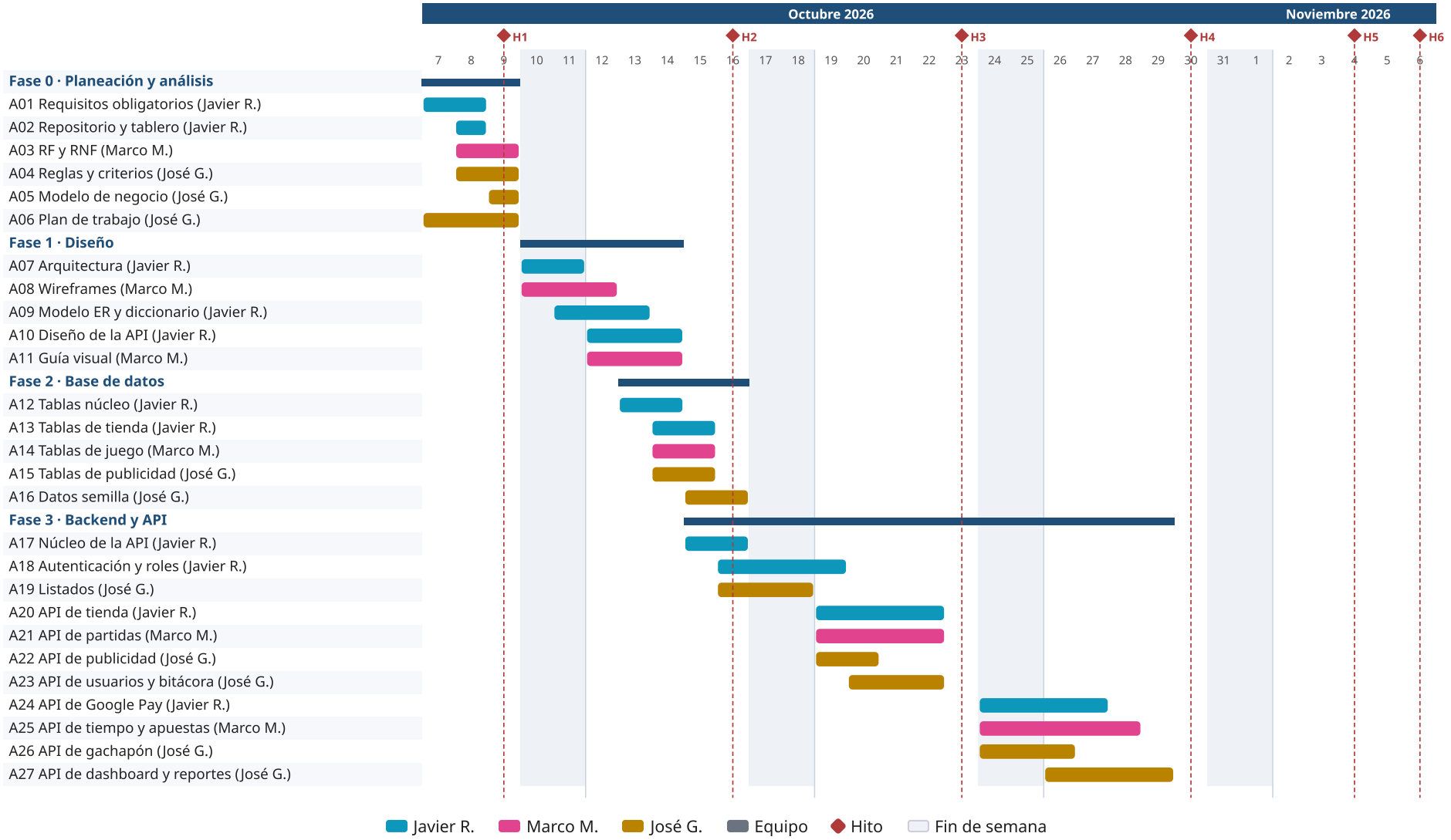


Figura 2. *Diagrama de Gantt por actividad (1/2): planeación, diseño, base de datos y backend. Color = responsable principal.*

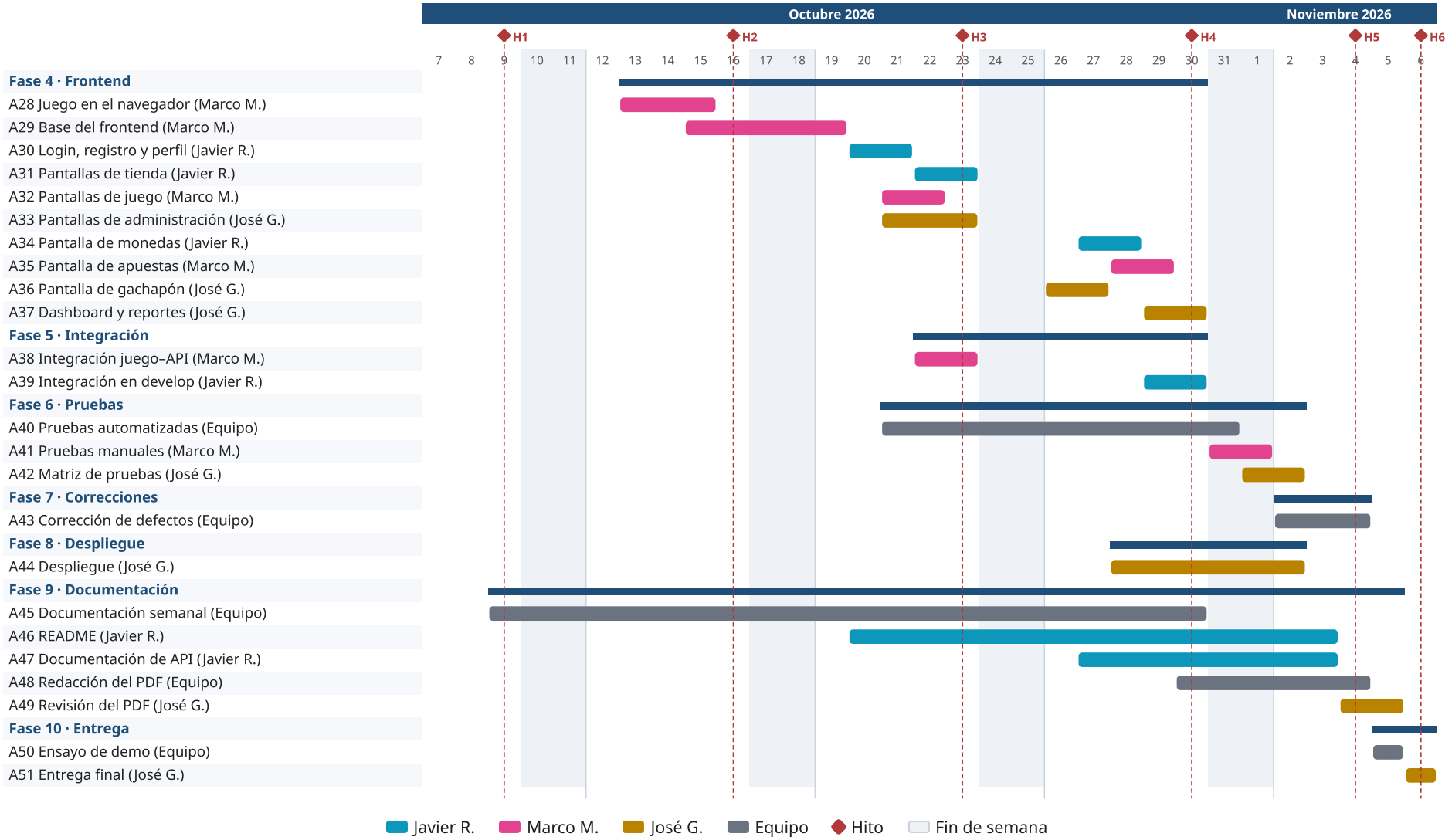


Figura 3. *Diagrama de Gantt por actividad (2/2): frontend, integración, pruebas, correcciones, despliegue, documentación y entrega.*

5. Recursos y decisiones técnicas

Tabla 8. Tecnologías, herramientas, equipos y servicios

Recurso	Elección	Justificación
Videojuego	Python 3.11, pygame-ce y pygbag (WebAssembly)	El juego ya existe en Python; pygbag lo ejecuta en el navegador sin reescribirlo, lo que respeta la jugabilidad original.
Frontend	HTML5, CSS3 y JavaScript con módulos ES; Chart.js para gráficas; anime.js para animaciones	No requiere compilación ni framework adicional; los módulos ES permiten componentes reutilizables (layout, tabla, CRUD, cliente de API). Chart.js cubre las gráficas dinámicas con poco código.
Backend	Python 3.11 con FastAPI, Uvicorn, PyMySQL, bcrypt, PyJWT y Pydantic	Mismo lenguaje que el juego. FastAPI valida las entradas con Pydantic, genera la documentación OpenAPI (Swagger) de forma automática y facilita aplicar autenticación y roles por endpoint. PyMySQL con SQL explícito da control de transacciones y bloqueos.
Gestor de base de datos	MariaDB 10.6+ (XAMPP en desarrollo; imagen oficial en Docker)	Motor relacional libre con InnoDB (claves foráneas, transacciones ACID), CHECK y vistas; compatible con MySQL; el equipo ya usa XAMPP en clase.
Pagos (simulados)	Google Pay API for Web, entorno TEST	Permite demostrar el flujo completo de pago sin cobrar dinero real.
Diseño	diagrams.net / Figma (wireframes); Mermaid (ER y secuencia)	Gratuitos; los diagramas en Mermaid se versionan como texto en el repositorio.
Pruebas	pytest con TestClient de FastAPI y una BD de pruebas aislada; DevTools del navegador para diseño responsivo	Pruebas repetibles de API, reglas y transacciones contra MariaDB real, sin alterar los datos de desarrollo.
Control de versiones y organización	Git + GitHub (ramas `main`, `develop` y `feature/*`, pull requests) y GitHub Projects	Historial de participación por integrante, revisión cruzada antes de integrar y tablero con una tarjeta por actividad.
Despliegue	Docker, Docker Compose, Nginx y Portainer (stack desde el repositorio de GitHub)	El mismo entorno en cualquier servidor; Nginx sirve el frontend y el juego y redirige `/api` a la API.
Equipos	3 computadoras personales (Windows) y un servidor con Docker y Portainer	Desarrollo local de cada integrante; el servidor aloja la versión desplegada.
Comunicación y editor	Microsoft Teams y chat del equipo; Visual Studio Code	Reuniones semanales y seguimiento diario; editor común con soporte para Python, JS y SQL.

Convenciones de trabajo

Commits con Conventional Commits (`feat`, `fix`, `test`, `docs`, `refactor`) y el ID de la actividad en el mensaje (por ejemplo, `feat(api): compra transaccional (A20)`). Una rama por actividad (`feature/A20-tienda`). Ningún cambio llega a `develop` sin la revisión de otro integrante.

6. Riesgos y seguimiento

6.1 Registro de riesgos

Tabla 9. Riesgos del proyecto

ID	Riesgo	Probabilidad	Impacto	Medida preventiva	Acción de respuesta	Responsable
RG1	El juego no funciona en el navegador o falla al conectarse con la API (tecnología nueva para el equipo)	Media	Alto	Compilar el juego con pygbag en la segunda semana (A28) para detectar problemas antes de integrar.	Ajustar una plantilla propia de pygbag; para la demo, usar el juego de escritorio mientras se corrige.	Marco M.
RG2	Participación desigual o commits concentrados en una persona	Media	Alto	Una tarjeta y una rama por actividad con responsable; revisar el historial de Git en cada reunión semanal.	Redistribuir actividades pendientes en la siguiente reunión y registrar el cambio.	José G.
RG3	Saldos inconsistentes por operaciones simultáneas o fallos a mitad de una operación	Baja	Alto	Transacciones con bloqueo de saldo (`SELECT … FOR UPDATE`) y CHECK del libro mayor; pruebas de ROLLBACK.	Corregir con prioridad máxima y volver a ejecutar las pruebas de transacciones.	Javier R.
RG4	Conflictos al integrar ramas al final	Media	Medio	Integrar a `develop` al menos dos veces por semana con pull requests pequeños.	Sesión de integración conjunta antes del hito más cercano.	Javier R.
RG5	Retrasos por carga académica de otras materias (exámenes y entregas)	Alta	Medio	Hitos semanales y holgura del 02/11 al 05/11; priorizar los requisitos obligatorios.	Posponer mejoras opcionales (animaciones y rediseño visual), nunca requisitos obligatorios.	José G.
RG6	Falla del despliegue en Docker/Portainer o servidor no disponible	Media	Medio	Iniciar el despliegue el 28/10 (A44), no al final; documentar la instalación local.	Demostración local con XAMPP siguiendo el README.	José G.
RG7	Servicios externos no disponibles (CDN de pygame-web, Google Pay, cdnjs)	Baja	Medio	Google Pay en modo TEST con opción de pago simulado; verificar los servicios antes de la demo.	Usar el pago simulado y el juego de escritorio durante la demostración.	Javier R.
RG8	Las apuestas y el gachapón podrían interpretarse como juego de azar con dinero real	Baja	Alto	Monedas virtuales sin valor, pagos solo en TEST, límites de juego responsable y probabilidades públicas.	Desactivar la compra de monedas fuera del entorno de pruebas.	Javier R.

6.2 Seguimiento del avance

- **Reuniones:** una reunión semanal de 30 minutos los lunes por Teams para revisar el avance contra este plan, los bloqueos y el historial de Git. Además, dos revisiones breves por chat (martes y jueves) con el formato “qué terminé, qué sigue, qué me bloquea”.
- **Herramienta de organización:** GitHub Projects con las columnas *Por hacer*, *En proceso*, *En revisión* y *Hecho*. Cada actividad A01–A51 es una tarjeta con responsable, fechas y criterio de terminado.
- **Revisión de entregables:** cada viernes (hitos H1–H4) el responsable muestra lo terminado y otro integrante lo verifica contra el criterio de terminado. Una actividad solo pasa a *Hecho* cuando su pull request se integra a `develop` con sus pruebas aprobadas.

- **Estados:** Planeado, En proceso, En revisión, Terminado, Retrasado y Cancelado.
- **Registro de cambios al cronograma:** las fechas de este documento son la **línea base** y no se sobrescriben. Todo cambio se anota en la bitácora de cambios con su justificación, y el seguimiento final compara la fecha planeada contra la fecha real de cada actividad (sección 5 del PDF final).

Tabla 10. Formato de la bitácora de cambios al cronograma

Fecha	Actividad	Fecha original	Fecha nueva	Motivo del cambio	Impacto en otras actividades	Aprobado por
dd/mm/aaaa	A##	dd/mm/aaaa	dd/mm/aaaa	(se llenará durante el desarrollo)	A## desplazada…	Equipo

7. Plan de pruebas y entrega

7.1 Estrategia de pruebas

Las pruebas de API se automatizarán con pytest y el TestClient de FastAPI contra una base de datos de pruebas aislada (`uma_racing_pruebas`) que se recrea desde `schema.sql` y `seed.sql` en cada ejecución. Cada responsable escribe las pruebas de sus módulos al terminar la actividad de backend correspondiente; José G. coordina la suite y la matriz de pruebas.

Tabla 11. Plan de pruebas

Tipo	Qué se comprobará	Cómo	Responsable	Cuándo	Evidencia esperada
Funciones principales	Caso exitoso de cada endpoint y de cada flujo de módulo	pytest + TestClient	Dueño del módulo	A20–A27	Pruebas en `backend/tests`
Permisos	401 sin token o con token revocado; 403 con rol incorrecto; recursos ajenos no visibles	pytest con cuentas de cada rol	Javier R.	A18, A40	Casos de permisos aprobados
Validaciones	Campos vacíos, formatos inválidos, rangos y campos no permitidos (422 con detalle); validación en formularios	pytest + prueba manual de formularios	Dueño del módulo	A40, A41	Casos de datos inválidos y capturas
Reglas de negocio	Al menos un caso que intente violar cada regla (saldo, límites diarios, antitrampa, último administrador, etc.)	pytest	Dueño del módulo	A40	Un caso por regla en la matriz
Transacciones	La compra, el pago, la apuesta y la tirada se aplican completas (COMMIT) o no cambian nada (ROLLBACK)	pytest comparando saldo e inventario antes y después	Javier R.	A20, A40	Casos de éxito y de fallo con saldo sin cambios
Integración interfaz–API–BD	Flujo punta a punta: registro → login → jugar → partida registrada → compra → historial y reportes actualizados	Prueba manual guiada sobre `develop` integrado	Marco M.	A39, A41	Capturas de cada paso y registros en la BD
Interfaz	Diseño responsivo (1366, 820 y 390 px), estados de carga, vacío, error y éxito, y confirmaciones	DevTools del navegador	Marco M.	A41	Capturas por resolución
Errores y seguridad	BD no disponible (mensaje sin trazas), inyección SQL, contraseñas con hash, datos inexistentes y duplicados	pytest + prueba manual con la BD detenida	José G.	A40, A41	Casos aprobados en la matriz
Aceptación	Criterios de aceptación de cada módulo	Demostración interna al congelar funciones	Equipo	30/10 (H4)	Lista de criterios verificada

Cobertura mínima planeada según los lineamientos: caso exitoso, datos inválidos, permisos insuficientes, registro inexistente, duplicidad, regla de negocio crítica, error de conexión o del servidor, operación transaccional y diseño responsivo. Los defectos se registrarán en la matriz de pruebas (A42) con su caso, su resultado y la corrección (A43).

7.2 Lista de verificación para la entrega final

- ☐ Sistema funcional (desplegado y en local) con todos los módulos integrados.
- ☐ Repositorio con historial de participación de los 3 integrantes.

- ☐ Ramas integradas en `develop` y `main` con la etiqueta `v1.0.0`.
- ☐ Scripts `schema.sql`, `seed.sql` y `seed_demo.sql` que crean la BD desde cero.
- ☐ Datos de prueba y cuentas para cada rol.
- ☐ `.env.example` y `.env` excluido del repositorio.
- ☐ Documentación de API (Swagger y `docs/api.md`).
- ☐ README completo con capturas.
- ☐ Instrucciones de instalación probadas en una computadora limpia.
- ☐ Suite de pruebas aprobada y matriz de pruebas con resultados.
- ☐ PDF de documentación con las 16 secciones de los lineamientos.
- ☐ Seguimiento del avance real contra este plan.
- ☐ Demostración ensayada con guion y datos de prueba.
- ☐ Entrega en Teams en la fecha indicada (06/11/2026).